Saarthi — Voice-Controlled Phone & Web Agent

Team DietCode · HackIIITD 2026 (Esya '26, IIIT Delhi) · Oct 17, 2026 · @Suyash

Abstract

Saarthi is a voice-first AI agent that operates a smartphone and the web on the user's behalf. The user speaks a goal in natural language (English, Hindi or Hinglish), and Saarthi plans the steps, taps, types and scrolls through real apps and websites, and confirms before any sensitive action. It targets people who find touchscreens and complex apps hard to use: elderly users, people with visual or motor impairments, first-time smartphone users, and anyone whose hands are busy.

Saarthi was built by Team DietCode for HackIIITD 2026 (Esya '26, IIIT Delhi) and was shortlisted after Round 1.

Problem Statement

Modern phones are powerful, but using them still demands precise touch input, literacy in English-heavy interfaces, and knowledge of where each feature hides. Existing voice assistants handle single commands (set an alarm, call Mom) but stop at the edge of third-party apps.

This leaves several groups underserved:

- **Elderly and first-time users** struggle with multi-step flows such as booking a ticket, paying a bill or ordering medicine.
- **People with visual or motor impairments** depend on screen readers, which are slow for complex, multi-screen tasks.
- **Regional-language users** find English-first app interfaces confusing.
- **Busy users** (driving, cooking, working) cannot use their hands safely.

The core gap: no widely available assistant can take a spoken goal and complete it end-to-end inside arbitrary apps and websites.

Proposed Solution

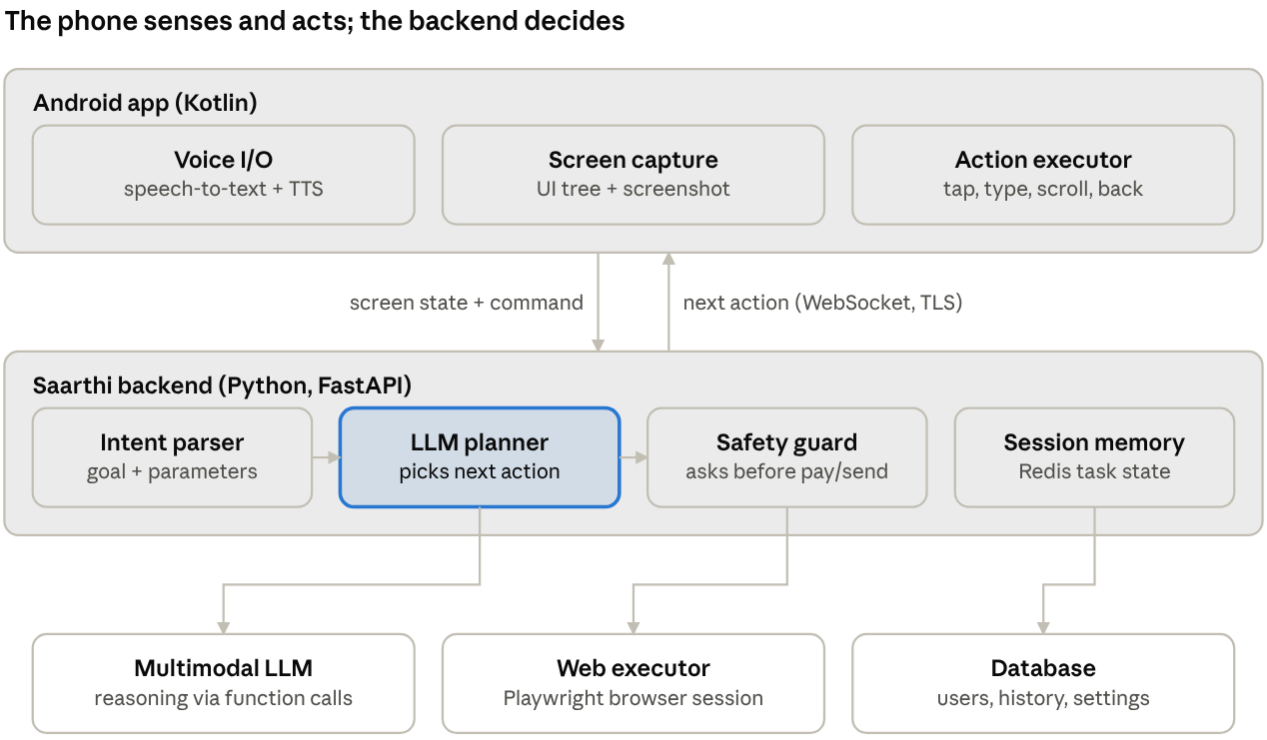
Saarthi turns a spoken goal into a sequence of real UI actions. It reads what is on screen, decides the next step with a large language model, performs that step, checks the result, and repeats until the goal is done.

Key Features

Feature	What it does
Voice-first control	Hands-free input with wake word or push-to-talk; replies spoken back via text-to-speech
Multilingual	Understands English, Hindi and Hinglish commands
Phone automation	Opens apps, taps, types, scrolls and navigates using Android Accessibility Service
Web automation	Completes tasks on websites through a controlled browser session
Screen understanding	Combines the accessibility tree (UI elements) with screenshots for visual context
Multi-step planning	Breaks a goal like "book a cab to Noida Sec 62" into ordered sub-steps
Self-correction	Verifies each action's outcome and retries or re-plans if the screen is unexpected
Human-in-the-loop	Asks for spoken confirmation before payments, sending messages or deleting data
Live feedback	Narrates what it is doing so the user always knows the current step

System Architecture

Saarthi splits work between a thin Android client and a cloud backend.



Saarthi system architecture · client, backend, services

The phone only listens, captures the screen and executes actions; all reasoning and safety checks run on the backend, which also drives a browser for web tasks.

Tech Stack

Layer	Technology
Mobile client	Android (Kotlin), Accessibility Service API, MediaProjection for screenshots
Speech-to-text	Whisper / Google Speech-to-Text (Hindi + English)
Text-to-speech	Android TTS / Google Cloud TTS
Reasoning (LLM)	Multimodal LLM (e.g. Gemini / GPT-4o / Claude) with function calling
Web automation	Playwright (headless Chromium)
Backend	Python, FastAPI, WebSockets for real-time action streaming
Session and state	Redis (short-term task memory)

Layer	Technology
Database	Firebase / PostgreSQL (users, task history, preferences)
Web dashboard	React + Tailwind CSS
Deployment	Docker on a cloud VM (GCP / AWS)

Workflow

Every task runs as a perceive → plan → act → verify loop until the goal is met or the user cancels.

1. **Listen:** The user says a goal, e.g. "WhatsApp pe Mom ko bolo main 7 baje aaunga."
2. **Transcribe:** Speech-to-text converts audio to text; language is auto-detected.
3. **Understand intent:** The LLM extracts the goal, target app and parameters (contact = Mom, message = "I'll come at 7").
4. **Perceive screen:** The client sends the current accessibility tree and a screenshot to the backend.
5. **Plan next action:** The LLM picks one action (open app, tap element, type text, scroll, go back) via function calling.
6. **Safety check:** If the action is sensitive (send, pay, delete), Saarthi reads it aloud and waits for "haan" / "yes".
7. **Act:** The Accessibility Service (phone) or Playwright (web) executes the action.
8. **Verify:** The new screen is captured and compared with the expected outcome; on mismatch, the agent re-plans.
9. **Report:** When done, Saarthi speaks a short confirmation: "Message sent to Mom."

Use Cases

User	Example command	What Saarthi does
Elderly parent	"Bijli ka bill bhar do"	Opens the payment app, finds the electricity biller, confirms amount aloud, pays after approval
Visually impaired user	"Read my last 3 emails"	Opens Gmail, reads sender and subject of each, offers to reply

User	Example command	What Saarthi does
Student	"Book the cheapest cab to Noida Sec 62"	Compares fares across ride apps, books after confirmation
Driver	"Navigate to the nearest petrol pump"	Opens Maps and starts navigation hands-free
Small shop owner	"Send today's offer to all customers on WhatsApp"	Drafts the message, reads it back, sends after approval
Any user (web)	"Fill this college form with my details"	Opens the site and fills saved fields, stops before submit

Security, Privacy and Safety

An agent that controls a phone must be trusted, so safety is built into every step.

- **Explicit confirmation:** Payments, sending messages, deleting data and form submission always need spoken approval.
- **Sensitive-field blocking:** The agent never reads or types passwords, OTPs, PINs or card numbers; the user enters these manually.
- **Data minimisation:** Screenshots are processed in memory and not stored; personal fields are masked before being sent to the LLM.
- **Encryption:** All client–server traffic uses TLS over secure WebSockets.
- **App allowlist:** Users choose which apps Saarthi may control; banking apps are off by default.
- **Kill switch:** Saying "stop" or pressing the floating button halts the agent instantly.
- **Audit log:** Every action is logged on-device so the user can review what was done.

Challenges and Limitations

- **Latency:** Each step needs a screen capture and an LLM call, so long tasks can take 10–30 seconds.
- **Dynamic UIs:** Pop-ups, ads and app updates can change layouts and break plans mid-task.
- **Accessibility gaps:** Some apps (games, custom-drawn views) expose little in the accessibility tree, forcing reliance on vision.
- **Platform limits:** iOS does not allow third-party apps to control other apps, so full automation is Android-only.
- **Cost:** Multimodal LLM calls per step add up at scale.

Future Scope

- On-device small language model for faster, offline, more private execution.
- Support for more Indian languages (Tamil, Bengali, Marathi, Telugu).
- Reusable "routines" learned from repeated tasks (e.g. a daily morning check).
- Caregiver mode, where a family member can remotely approve sensitive actions.
- Integration with government services (DigiLocker, UMANG) for citizen assistance.
- A developer SDK so apps can expose clean actions directly to Saarthi.

Team DietCode

Member	Role
Suyash Shukla	
Suyash Dwivedi	
Piyush Tiwari	
Aabhas Viswas	

Conclusion

Saarthi moves voice assistants from answering questions to getting real work done inside any app or website. By pairing multimodal LLM reasoning with Android accessibility and browser automation, and keeping the user in control of every sensitive step, it makes smartphones usable for millions who are left out by touch-first, English-first design.